Lab Exercise: Bank Account ADT Implementation

Instructions

Complete the implementation of the `BankAccount` Abstract Data Type (ADT) and client application below. Fill in every missing code section marked with `/* TODO 1 */` through `/* TODO 11 */`.

Part 1: Header File (`BankAccount.h`)

Fill in access specifiers and constant function qualifiers for the class definition.

```
/**
 * @file BankAccount.h
 * Class specification for BankAccount ADT
 */
#ifndef BANKACCOUNT_H
#define BANKACCOUNT_H

#include <iostream>
#include <string>

class BankAccount {
/* TODO 1: Add access specifier for private data members */
----------------------

    std::string accountNumber;
    std::string accountHolder;
    double balance;

/* TODO 2: Add access specifier for public member functions */
----------------------

    // Default Constructor
    BankAccount();

    // Parameterized Constructor
    BankAccount(std::string accNum, std::string holder, double initialBalance);

    // Destructor
    ~BankAccount();

    // Mutator functions
    void deposit(double amount);
    bool withdraw(double amount);

    // TODO 3: Declare getBalance() and display() as CONST member functions
    double getBalance() ______________________;
    void display() ______________________;
};

#endif
```

Part 2: Implementation File (`BankAccount.cpp`)

Complete constructor initializations, scope resolutions, and logic implementations.

```cpp
/**
 * @file BankAccount.cpp
 * Implementation of BankAccount ADT
 */
#include "BankAccount.h"
#include <iostream>

// Default Constructor initialization
BankAccount::BankAccount()
    : accountNumber("0000"), accountHolder("N/A"), balance(0.0) {
}

// TODO 4: Complete the Parameterized Constructor implementation
BankAccount::BankAccount(std::string accNum, std::string holder, double initialBalance)
    : accountNumber(accNum), accountHolder(holder) {
    if (initialBalance >= 0.0) {
        /* Set balance member variable to initialBalance */
        ____________________________________________;
    } else {
        /* Set balance to default 0.0 when negative balance provided */
        ____________________________________________;
    }
}

// Destructor
BankAccount::~BankAccount() {
}

void BankAccount::deposit(double amount) {
    if (amount > 0) {
        balance += amount;
        std::cout << "Deposited $" << amount << " successfully.\n";
    } else {
        std::cout << "Invalid deposit amount!\n";
    }
}

// TODO 5: Complete the withdraw member function header with scope resolution
bool ______________________::withdraw(double amount) {

    // TODO 6: Complete condition checking if amount is positive AND balance is sufficient
    if (__________________________________________________) {
        balance -= amount;
        std::cout << "Withdrew $" << amount << " successfully.\n";
        return true;
    } else {
        std::cout << "Withdrawal failed: Insufficient funds or invalid amount.\n";
        return false;
    }
}

// TODO 7: Complete getBalance scope resolution and return statement
double ______________________::getBalance() const {
    return ______________________;
}
```

```
void BankAccount::display() const {
    std::cout << "----------------------------\n";
    std::cout << "Account Number : " << accountNumber << "\n";
    std::cout << "Account Holder : " << accountHolder << "\n";
    std::cout << "Current Balance: $" << balance << "\n";
    std::cout << "----------------------------\n";
}
```

Part 3: Client Application (`main.cpp`)

Complete function signature for pass-by-const-reference and array operations.

```
/**
 * @file main.cpp
 * Client file to demonstrate BankAccount ADT
 */
#include <iostream>
#include "BankAccount.h"

// TODO 8: Complete parameter type to receive BankAccount object by CONST REFERENCE
void printAccountSummary(_____________________________________ account) {
    std::cout << "\n[Summary via Const Reference Parameter]\n";
    account.display();
}

int main() {
    // TODO 9: Declare an array named 'accounts' storing 3 BankAccount objects
    ____________________ accounts[3] = {
        BankAccount("ACC-101", "Alice Smith", 500.0),
        BankAccount("ACC-102", "Bob Jones", 1200.50),
        BankAccount("ACC-103", "Charlie Brown", 250.75)
    };

    std::cout << "=== INITIAL ACCOUNT STATES ===\n";

    // TODO 10: Complete loop calling display() for each object in the array
    for (int i = 0; i < 3; ++i) {
        accounts[i].____________________();
    }

    std::cout << "\n=== PERFORMING TRANSACTIONS ===\n";

    // Deposit into the first account
    accounts[0].deposit(150.0);

    // Withdraw from second account
    accounts[1].withdraw(300.0);

    // TODO 11: Pass the first account to printAccountSummary
    printAccountSummary(____________________);

    return 0;
}
```